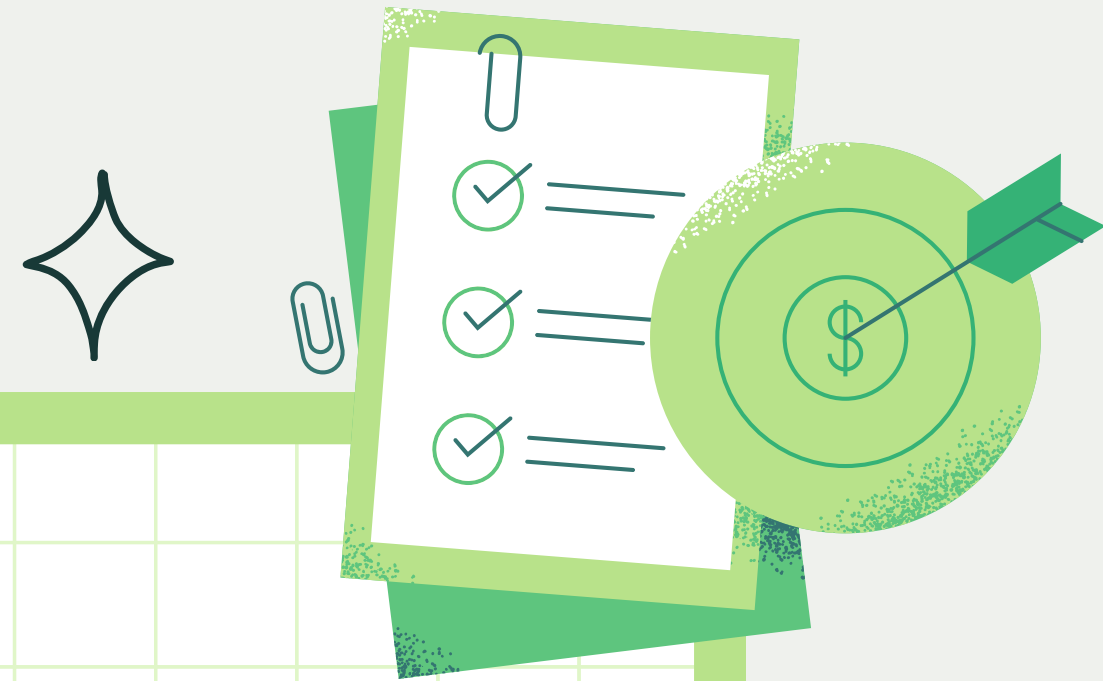
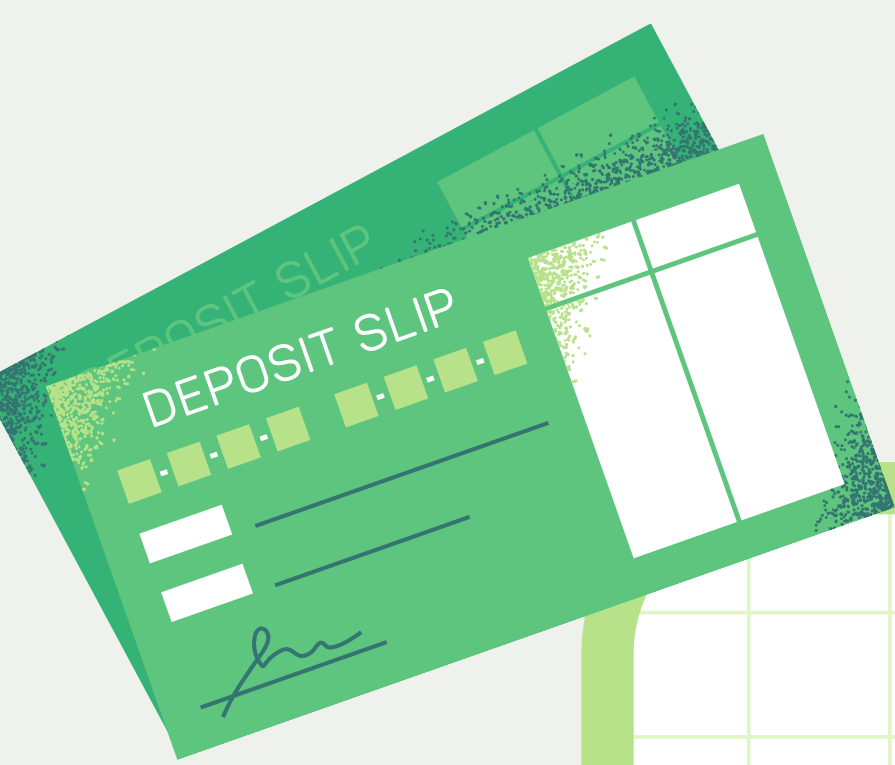
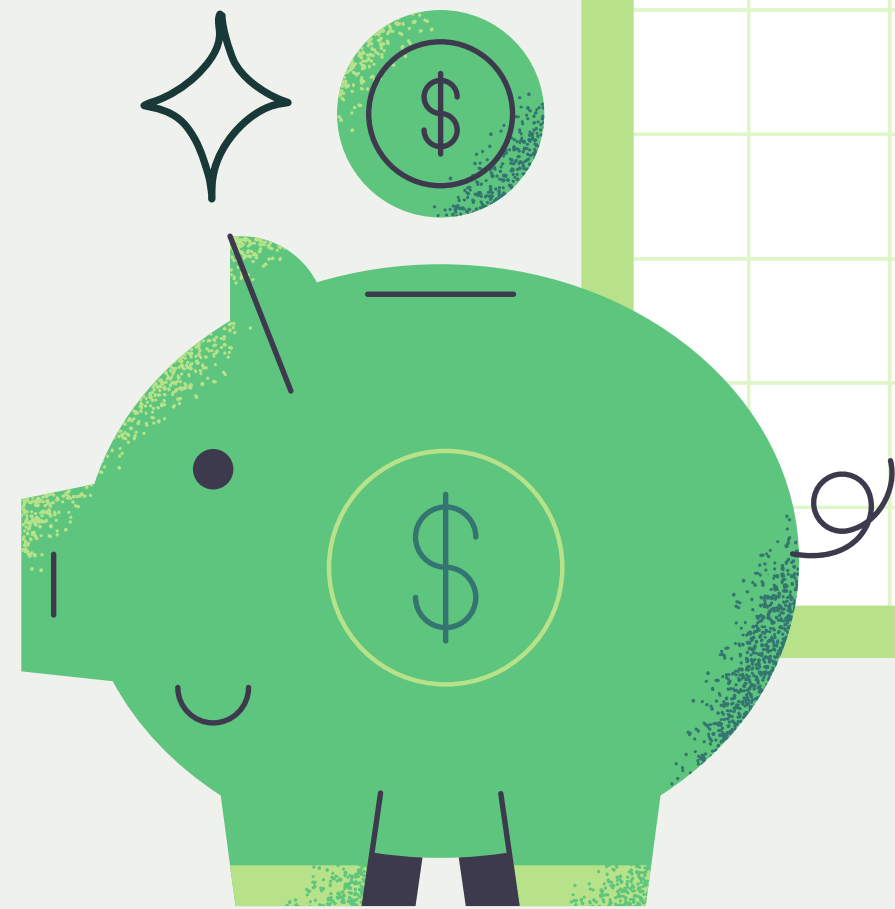




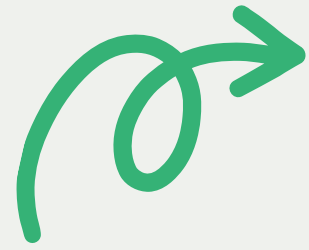
Banking Management System

(MultiUser)



Project by Harini.K



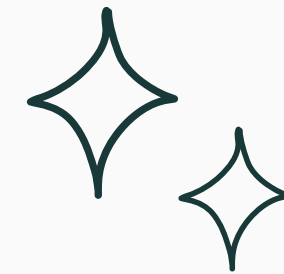


PROJECT OVERVIEW



- 01. Python based project - Simple banking application using Python
- 02. Python application - developed using basic python concepts
- 03. Console system -Runs in IDLE/CMD, masking works properly in CMD
- 04. Multi user system - Supports multiple users with separate accounts
- 05. Multi user support - Allows many users to perform various operations
- 06. File handling/storage - Data stored permanently in a text file

OBJECTIVES



- 01. Build banking system - Create a working banking model
- 02. CRUD operations - Perform create, read, update, and delete on user data
- 03. Authentication - Ensure secure login using username and password verification
- 04. Transaction tracking - Record and display all user transactions clearly
- 05. User friendly design - Provide simple and easy to use menu driven interface
- 06. Data management - Store and retrieve user data efficiently using file handling

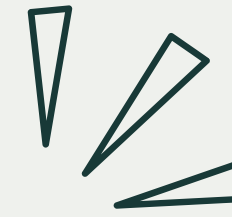


TECHNOLOGIES USED ↻



Python is used as the main programming language to develop the system. File handling is used to store and retrieve user data from a text file. The msvcrt module is used for password masking (in CMD) by displaying '*' instead of actual characters, while IDLE uses normal input due to compatibility limitations. A dictionary is used to store user data in key-value format for efficient access and updates.





KEY FEATURES



Accounts

This feature allows users to create new accounts easily and provides a secure login system for both users and admin to access their accounts safely.



Transactions

This feature enables users to deposit and withdraw money as well as transfer funds between accounts, ensuring smooth and efficient money operations.

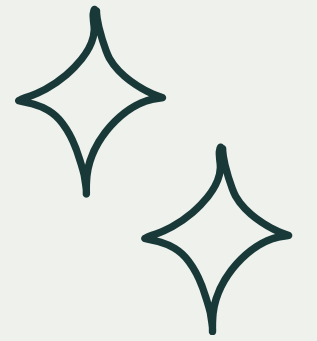


Tracking

This feature records all transactions and allows users to view transaction history for tracking account activity. Users can also view account balance anytime.



ADVANCED FEATURES



Interest calculation

Calculates interest on balance

Loan eligibility

Checks if user qualifies for loan

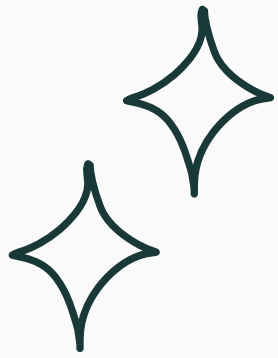
Admin panel

Admin manages all accounts

Delete account

Users/admin can remove accounts





SYSTEM DESIGN



Dictionary storage - Stores user data in memory for fast access

File storage - Saves data permanently in bank.txt

Functions - Handles each operation using separate functions

Menu interface - Provides simple user interaction through options

Data loading & saving - Loads data at start and saves after each operation



SECURITY FEATURES



- **Password Masking**

Password is hidden using * while typing to prevent others from seeing it.

- **Authentication**

System verifies username and password before allowing access.

- **Admin Control**

Admin has full access to manage and control all user accounts.



LIMITATIONS

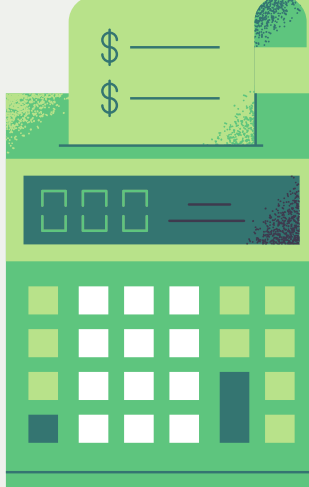


Data stored in text file - not scalable

Works for small data, but becomes slow and hard to manage with many users.

Password masking issue - in IDLE

Masking works in CMD using msvcrt, but IDLE has limitations.



Console UI - Limited Interaction

Simple and user-friendly, but lacks graphical interaction.

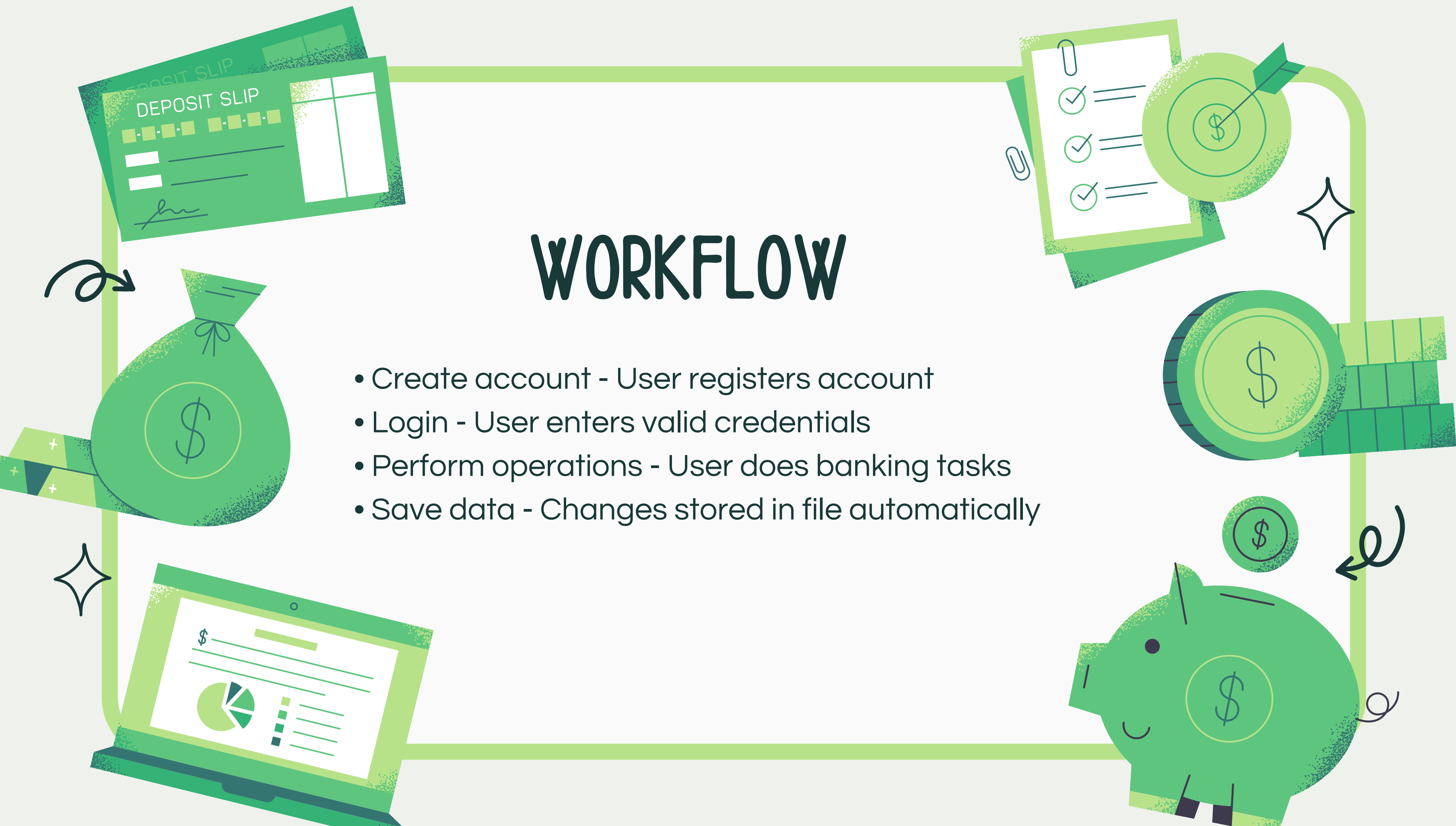
No Database - Slow for Large Data

Data handling becomes inefficient as data size increases.



WORKFLOW

- Create account - User registers account
- Login - User enters valid credentials
- Perform operations - User does banking tasks
- Save data - Changes stored in file automatically



SYSTEM DEMONSTRATION

```
1. Create Account
2. Login
3. Exit
Enter choice: 1
Create username: s129
Create password: *****
Enter initial balance: 500
Account created successfully!
```

Step 1: Create Account

- Enter user details
- Initial balance setup

```
1. Create Account
2. Login
3. Exit
Enter choice: 2
Enter username: s129
Enter password: *****
User Login Successful
```

Step 2: Login

- Enter credentials
- Successful authentication

```
- USER MENU -
1. View Balance
2. Deposit
3. Withdraw
4. Transfer
5. Transaction History
6. Interest
7. Loan Check
8. Delete Account
9. Logout
Enter choice: 1
Current Balance: 500
```

Step 3: Process Output

- Select operation
- Displays result (ex: Balance)



FUTURE IMPROVEMENTS



- **Graphical User Interface (GUI)**

- Develop an interactive interface using Tkinter for better user experience
- Builds forms and UI for interactive, user-friendly experience

- **Secure Password System**

- Implement encryption techniques to protect user credentials.
- It secures passwords in coded format
- Makes passwords unreadable even if data is accessed

- **Database integration [MYSQL]**

- MySQL improves storage, speed, and data organization
- Faster data access and processing with efficient retrieval

- **Mobile/Web Expansion**

- Currently works only on a computer
- Can be upgraded to a web application (using HTML, CSS, Flask/Django)
- And mobile app (using Android or cross-platform tools)



CONCLUSIONS



- A simple yet effective banking simulation system
- Covers essential banking functionalities
- Helps beginners understand real-world application of Python
- Demonstrates concepts like file handling, functions, and authentication





THANK YOU

- Harini.K

